

Rental Management

A rental system for small landlords, with a tenant portal

Lidor Amraby · final project · rental-management-app-wine.vercel.app

What it is

- Buildings and the units inside them
- Tenancies: who rents what, for how long, at what rent
- A rent ledger the landlord writes as money arrives
- Repairs, reported by the tenant and followed to the end

It records money received. It is not a payment processor.

The problem

A spreadsheet, a phone, and a memory.

- What is owed is arithmetic somebody does by hand
- The tenant cannot check anything without asking
- A repair promised by phone leaves no record

The users

- **Landlord** — owns the portfolio, records everything
- **Tenant** — reads their own tenancy and its ledger
- A tenant has two writes: report a problem, confirm a repair

Neither of them is rent. Only the landlord records money.

The business value

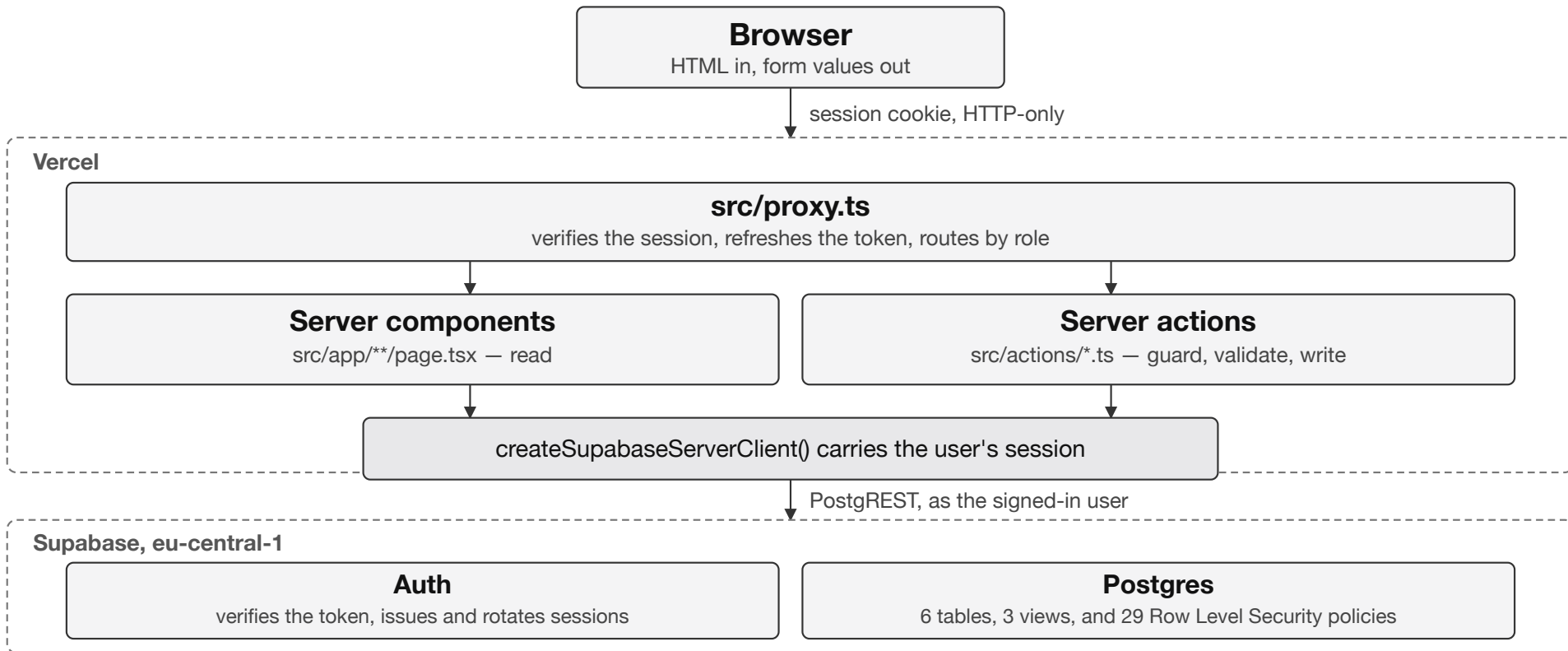
- Arrears visible at a glance, always current
- Fewer phone calls: the tenant sees their own position
- Every repair has a timestamped route and a confirmation

How it is built

- Next.js App Router, TypeScript, deployed on Vercel
- Supabase: Postgres, Auth, Row Level Security
- Server components read. Server actions write.
- No database client in the browser at all

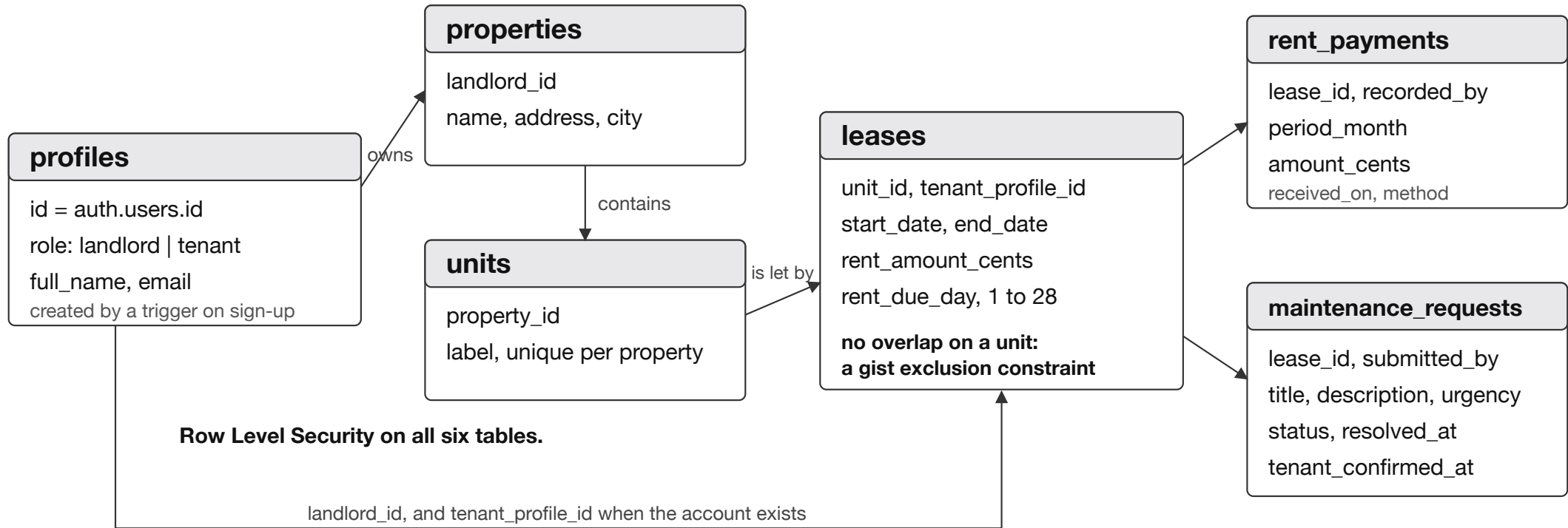
Authorisation lives in the database, not in the application.

The architecture



The routing is a convenience. The database is the boundary that refuses.

The database



No lease status column, no occupancy flag, no rent periods table. All derived.

Every write, the same seven steps

- 1 Resolve the acting user from the session
- 2 Refuse anyone who is not the right role
- 3 Parse the input with the form's own schema
- 4 Apply the business rules that need other rows
- 5 Write, with Row Level Security as the last word
- 6 Revalidate the pages that changed
- 7 Return a typed result the form can render

The landlord id comes from the session, never from the form.

Demo

`rental-management-app-wine.vercel.app`

Including two deliberate failures: an overlapping tenancy, and one tenant reaching for another's data

The tests

- 325 unit and component tests — the rules at their boundaries
- 98 permission and database tests — against a real Postgres
- 15 end-to-end tests — whole processes in a browser
- 5 documented manual checks — print, layout, screen reader

The permission tests attack the database, not the interface.

Scale

Measured against synthetic portfolios, not assumed.

- Tens of landlords: every page under about 120 ms of database time
- Hundreds: two problems, both measured and priced
- Functions run in Washington, the database is in Frankfurt
- 98 ms → 314 ms purely because another landlord's rows exist

Security

- Session in one HTTP-only cookie, against the library default
- 29 Row Level Security policies, proved by 98 tests
- Validation always runs on the server
- Service role key: one caller, unreachable from the browser

Still missing, and written down: rate limiting, MFA, an audit log.

With more time

- 1 Move the functions to the database's region
- 2 Give two aggregate queries an indexable filter
- 3 Halve the fixed round trips per request
- 4 More than one person on a portfolio
- 5 An audit log, and rate limiting on my own endpoints

In that order: measured effect over risk.

Thank you

Deployed. Tested at three levels. Every decision written down.

rental-management-app-wine.vercel.app · github.com/lidorStudent/rental-management-app